

UrbanPulse - Collaborative Urban Issue Tracker

High-Level System Architecture — Component & Communication Description

1. System Overview

UrbanPulse is a distributed, event-driven platform that allows citizens to report local infrastructure problems (potholes, broken lights, graffiti) via a mobile app. GPS coordinates automatically route each report to the responsible regional server. A microservices architecture separates concerns across eight independently deployable components, connected through a combination of synchronous REST/gRPC calls and asynchronous RabbitMQ events. An AI classification service automatically categorises reports by type and urgency and detects duplicates using a multimodal vision model.

2. Components — Platform, Language & Role

Each component below can be deployed on a distinct server or container. The table describes the technology stack and primary responsibility of each.

Component	Platform / Language	Role & Responsibilities
Mobile App	React Native (iOS & Android) TypeScript / Expo SDK 51	Primary citizen-facing interface. Allows users to submit issue reports with photos, text, and auto-captured GPS location. Provides push notification feedback on report.
Web UI	React Native Web TypeScript, served via Nginx	Administrator and supervisor dashboard. Enables triaging, assigning, and tracking resolution of reports. Connects to the API Gateway via HTTPS REST and receives real-time updates via WebSockets.
API Gateway	Node.js 22 / Express 5 TypeScript	Single entry point for all client traffic. Handles TLS termination, rate limiting, request routing, and JWT validation by delegating to the Identity Server. Forwards authenticated requests to internal services.
Identity Server	.NET 10 / C# Duende IdentityServer 7 PostgreSQL 16 for user store	OpenID Connect / OAuth 2.0 provider built on Duende IdentityServer. Manages user registration, login, role assignment (citizen, contractor, supervisor).
Reports Server	Java 21 / Spring Boot 3.3 Spring Data JPA + PostGIS	Core domain service. Exposes a CRUD REST API for report lifecycle management (create, read, update, close). Uses PostGIS for GPS-based zone routing and geospatial queries.

Component	Platform / Language	Role & Responsibilities
AI Classification Server	Python 3.12 / FastAPI 0.111 OpenAI Vision API (GPT-4o) openai-python SDK 1.x	Consumes <i>report.created</i> events, sends photo and text to the OpenAI Vision API for classification (category, urgency level, duplicate detection), then publishes a
Notification Service	.NET 10 / C# SendGrid SDK v9 (email)	Consumes <i>report.status_changed</i> and <i>user.registered</i> events and sends transactional emails via the SendGrid HTTP API. Responsible only for outbound email; future channels (SMS
Message Broker	RabbitMQ 3.13 AMQP 0-9-1 protocol	Durable, async event bus decoupling producers from consumers. Hosts exchanges/queues for: <i>report.created</i> , <i>report.status_changed</i> , <i>report.classified</i> , <i>user.registered</i> . Guarantees at least

Table 1 — Deployable components with platform and role

3. Communication — Protocols, Patterns & Technologies

3.1 Synchronous Communication (REST / gRPC)

All client-facing traffic flows through the **API Gateway** over HTTPS (TLS 1.3). The gateway performs JWT validation by calling the **Identity Server** via the OpenID Connect introspection endpoint, then proxies the request to the appropriate internal service over plain HTTP on the private network. The **Web UI** additionally maintains a persistent **WebSocket** connection through the gateway to receive real-time report-status updates without polling.

Cross-zone synchronisation between regional **Reports Server** instances uses **gRPC** (Protocol Buffers v3) over mutual TLS. This bidirectional streaming connection supports both data-replication (propagating new reports) and control messages (requesting zone reassignment), making it the backbone of the multi-zone coordination layer.

3.2 Asynchronous Communication (Event Bus)

Loose coupling between backend services is achieved through **RabbitMQ 3.13** using the **AMQP 0-9-1** protocol. Producers publish events to named exchanges without knowledge of consumers; RabbitMQ routes messages to bound durable queues, guaranteeing at-least-once delivery via publisher confirms and consumer acknowledgements.

- **report.created** — published by the Reports Server after a new report is persisted; consumed by the AI Classification Server.
- **report.classified** — published by the AI Classification Server after model inference; consumed by the Reports Server to update the record.
- **report.status_changed** — published by the Reports Server on any status transition; consumed by the Notification Service.
- **user.registered** — published by the Identity Server (via a Duende IdentityServer event sink) after successful user creation; consumed by the Notification Service.

3.3 External Service Integrations

The **AI Classification Server** makes synchronous HTTPS calls to the **OpenAI Vision API** (GPT-4o) within its Kafka consumer loop, sending the report photo and text as a multimodal prompt and receiving a structured JSON response with category, urgency score, and duplicate flag. The **Notification Service** delivers transactional emails through the **SendGrid API** via the official .NET SendGrid SDK v9, using API-key authentication.

4. Connections & Protocol Summary

Connection	Protocol / Technology	Purpose & Pattern
Clients → API Gateway	HTTPS / REST (TLS 1.3) WebSocket (wss://)	Synchronous request-response for CRUD operations and auth. WebSocket for real-time status push to Web UI
API Gateway → Identity Server	HTTPS / OpenID Connect JWT introspection	Token validation and user-info retrieval on every authenticated request (control protocol)
API Gateway → Reports Server	HTTP/1.1 REST (internal) JSON payloads	Proxied CRUD calls for report management. Internal network only, no TLS overhead
Reports Server → Message Broker	AMQP 0-9-1 / RabbitMQ Spring AMQP (spring-rabbit 3.x)	Publishes report.created and report.status_changed events to RabbitMQ exchanges. Async, publisher confirms enabled
Identity Server → Message Broker	AMQP 0-9-1 / RabbitMQ RabbitMQ.Client .NET SDK 6.x	Publishes user.registered events after successful registration.
AI Classification Server ← Message Broker	AMQP 0-9-1 / RabbitMQ aio-pika 9.x (async Python)	Subscribes to report.created queue. Pulls messages, calls OpenAI API, publishes report.classified. Pattern: event-driven pipeline
Reports Server ← Message Broker	AMQP 0-9-1 / RabbitMQ Spring AMQP consumer	Consumes report.classified to update the report record with AI-assigned category and urgency
Notification Service ← Message Broker	AMQP 0-9-1 / RabbitMQ RabbitMQ.Client .NET SDK 6.x	Consumes report.status_changed and user.registered to trigger outbound emails.
Notification Service → SendGrid	HTTPS / SendGrid Web API v3 REST + API key auth	Reporting protocol: delivers transactional email notifications to citizens and
AI Classification Server → OpenAI	HTTPS / OpenAI REST API openai-python SDK 1.x	Sends image + text payload; receives structured classification JSON. Synchronous call within async consumer loop
Reports Server ↔ Reports Server (multi-zone)	gRPC (Protobuf) over mTLS grpcio 1.63	Data replication and cross-zone report synchronization. Bi-directional streaming for zone handoff

Table 2 — Inter-component connections with protocol type and communication pattern